Guión de trabajo tutorizado

Bases de datos In-Memory I

Gestión y visualización de datos masivos

Profesor Dr. Domingo Savio Rodríguez Baena
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA-BasesDeDatosInMemoryI-Actividad.pdf» (Campus Virtual UPO).

Objetivos

• Afianzar los conceptos clave asociados a las bases de datos In-Memory.

• Investigar sobre ejemplos concretos de tecnología y casos de uso.

Ejercicios/Problemas

E1. (60 minutos – 1.5 punto) Lea el artículo cuya URL aparece en la diapositiva 48 y exponga de forma clara un resumen del mismo y sus conclusiones personales al respecto.

E2. (90 minutos – 3 puntos) Las bases de datos In-Memory no son una tendencia moderna, sino que ya se empezaban a estudiar en la década de los 80. Sin embargo, podemos encontrar diferencias interesantes entre los planteamientos y las necesidades de aquella época y la actual. Lea el artículo del año 1992 Main Memory Database System: An Overview y responda a estas preguntas:

2.1 ¿De qué forma se deja patente en el artículo cómo el hardware de la época era una limitación a las distintas aplicaciones de las bases de datos In-Memory?

2.2 ¿Qué diferencias, según los autores, existe entre una base de datos In-Memory de aquella época con una base de datos convencional con un gran cache?

2.3 ¿Qué era necesario mantener de los sistemas convencionales para evitar pérdidas de datos ante fallos de sistema de las MMDB?

2.4 ¿Alguna de las bases de datos In-Memory comentadas en el artículo han sobrevivido hoy en día? Concluya este ejercicio con sus propias conclusiones acerca de lo que ya era una idea futura en aquellos años.

E3. (60 minutos – 1,6 puntos). Investigue un caso de uso y de éxito real y lo más actual posible de las bases de datos In-Memory y haga un resumen acerca del mismo: empresa, tipo de plataforma escogida, objetivos, mejoras conseguidas, objetivos futuros, etc.

E4. (60 minutos – 1,4 puntos). Desde la versión Enterprise Server 3.26, MONGODB incluye un motor IN-MEMORY totalmente integrado.

Investigue cómo poder activar la opción IN-MEMORY usando la instalación de MongoDB que tuvo que llevar a cabo en la actividad de la sesión de Bases de Datos Documentales y describa el proceso de la forma más clara posible.

E5. (90 minutos – 2,50 puntos). La base de datos REDIS es una de las bases de datos In-Memory más utilizada. De tipo CLAVE-VALOR y solo disponible para LINUX y OS X, es fácil de usar y proporciona un enorme rendimiento.

Lleve a cabo la instalación de REDIS en su ordenador, mostrando los pasos realizados y comentando las posibles decisiones o dificultades experimentadas.

Tanto la base de datos como la guía de instalación están en https://redis.io/download.

Recuerde que va a necesitar LINUX.

Fecha y Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- La entrega se ha de llevar a cabo durante el día 18 de mayo de 2026.

Bibliografía

1. Main Memory Database Systems: An Overview. IEEE Transactions on Knowledge and Data Engenieering. December 1995.

2. Mongodb (https://docs.mongodb.com/manual/core/inmemory/)

3. REDIS (https://redis.io/)

RESOLUCIÓN

E1. Resumen del artículo de la diapositiva 48 y conclusiones

La diapositiva 48 (Microsoft, *In-Memory OLTP / Project Hekaton*) enlaza el artículo **«Why I don't (yet) recommend In-Memory OLTP to my customers»** de Klaus Aschenbrenner (SQLpassion, 11 de marzo de 2015), consultor especializado en rendimiento de SQL Server.

Resumen

El autor reconoce que In-Memory OLTP (tablas optimizadas para memoria y procedimientos compilados de forma nativa, introducidos en SQL Server 2014) es una tecnología «impresionante». Aun así, explica por qué **todavía no la recomienda** a sus clientes: es una versión 1.0 con demasiadas restricciones, y sus clientes toman decisiones tecnológicas a unos cinco años vista. Sólo recomienda algo que encaje al 100 % en su entorno. Las limitaciones que enumera son:

- **Sin claves foráneas ni restricciones CHECK**: no se puede garantizar la integridad referencial, lo que impide migrar bases de datos OLTP existentes (lo considera un *show-stopper*).
- **Sin cambios de esquema**: no admite `ALTER TABLE` ni añadir índices después de crear la tabla. Cambiar el número de *buckets* de un índice hash obliga a borrar y recrear la tabla.
- **Sin planes de ejecución** en los procedimientos compilados de forma nativa: se ejecuta código máquina, así que no hay plan real que analizar cuando surge un problema de rendimiento.
- **Sin recompilaciones**: los procedimientos no se recompilan si cambia la distribución de los datos, y no existe `ALTER PROCEDURE`; hay que borrarlos y recrearlos.
- **Lógica de negocio costosa en CPU**: mover lógica intensiva al motor sale caro con licencias Enterprise de unos 7.000 USD por núcleo.
- **Mucha RAM**: Microsoft recomienda el doble del tamaño de las tablas, más la memoria necesaria para las versiones de fila del control de concurrencia optimista.
- **Uso fuera de su propósito**: desaconseja emplearlo para *staging* de ETL o como sustituto de TempDB. Es una tecnología pensada para OLTP.

Concluye invitando a Microsoft a eliminar estas limitaciones y se compromete a revisar su postura cuando lo haga. De hecho, el mismo autor publicó en noviembre de 2015 «Why I now (eventually) recommend In-Memory OLTP» tras las mejoras de SQL Server 2016: soporte de claves foráneas y restricciones CHECK, índices sobre columnas de texto sin intercalación BIN2, `ALTER TABLE` y reconstrucción de índices para cambiar el número de *buckets*.

Conclusiones personales

El artículo muestra que **una base de datos en memoria no es sólo «lo mismo pero más rápido»**. Guardar los datos en RAM obliga a rediseñar estructuras (índices hash, índices sin *latches*), concurrencia (control optimista multiversión) y compilación. Esos cambios tienen un coste de funcionalidad y de operación. Coincide con la idea de la diapositiva: Microsoft optó por una integración *without disruption*, en el mismo producto y moviendo a memoria sólo

tablas concretas. Eso permite adoptarla de forma selectiva en las tablas calientes, pero en la primera versión el precio era perder parte del modelo relacional. La lección para un proyecto real es evaluar la tecnología por su **madurez y encaje** (integridad, mantenimiento, monitorización y coste de memoria y licencias), no sólo por el *benchmark*. El propio autor cambió de opinión en pocos meses, y hoy la funcionalidad está integrada también en Azure SQL. Eso ilustra el ciclo habitual de estas tecnologías: primero una versión muy restrictiva y después la maduración.

E2. Main Memory Database Systems: An Overview (García-Molina y Salem)

El artículo (IEEE *Transactions on Knowledge and Data Engineering*, vol. 4, nº 6, diciembre de 1992) repasa cómo afecta a cada componente de un SGBD que la copia primaria de los datos resida permanentemente en memoria (MMDB) en lugar de en disco (DRDB): concurrencia, confirmación de transacciones, representación de los datos, métodos de acceso, procesamiento de consultas y recuperación. Termina con una revisión de siete sistemas de la época.

2.1 ¿Cómo muestra el artículo que el hardware de la época limitaba las aplicaciones de las MMDB?

- **Capacidad y precio de la memoria.** La primera pregunta que plantea es si es razonable suponer que la base de datos cabe entera en memoria. La respuesta es «sí, para algunas aplicaciones»: bases pequeñas o que crecen más despacio que la memoria disponible (p. ej. tiempo real en telecomunicaciones, radar o *trading*). Para bases grandes (imágenes de satélite) propone **particionar** en datos «calientes» en memoria y datos «fríos» en disco, como IMS con Fast Path. La RAM no daba para todo.
- **Volatilidad.** La memoria principal es volátil y, aunque puede hacerse más fiable con baterías, SAI, memoria con corrección de errores o redundancia modular, nunca es fiable al 100 %. Además, al ser accesible directamente por el procesador, es más vulnerable a errores de software que el disco. Hay que mantener copias en disco y registros.
- **Coste de escribir en disco.** Escribir en disco cuesta mucho más que escribir en memoria. El *log* y los *checkpoints* se convierten en el cuello de botella, y buena parte de las técnicas descritas (grupos de commit, pre-commit, memoria estable para la cola del log) intentan esquivarlo.
- **Hardware específico.** Varios diseños recurren a hardware dedicado para compensar: MARS usa un procesador de recuperación y memoria no volátil, y HALO propone un dispositivo que intercepta los accesos a memoria para generar el log. Con el hardware estándar no bastaba.
- **La regla de los 5 minutos** (Gray y Putzolu), citada en la conclusión: con los precios de entonces sólo compensaba tener en memoria los datos accedidos al menos cada 5 minutos. Los autores prevén que el umbral subirá conforme baje el precio de la memoria, lo que confirma que el factor limitante era económico y tecnológico.

2.2 ¿Qué diferencias hay entre una MMDB y una base de datos convencional con una caché muy grande?

Aunque la caché contenga todos los datos, una DRDB **no aprovecha que están en memoria**, porque sus estructuras están diseñadas para disco:

- Los **índices** son de tipo B-tree, optimizados para accesos por bloques, cuando en memoria son más eficientes estructuras como los T-trees o el *hashing*.
- Cada acceso pasa por el **gestor de buffers**: calcular la dirección de disco de la tupla, comprobar si el bloque está en caché y copiarlo al buffer de la aplicación. En una MMDB se accede directamente por la **dirección de memoria** del objeto.
- La MMDB puede usar **punteros** en la representación de los datos (p. ej. atributos de longitud variable apuntando a un *heap*, índices que apuntan a las tuplas sin almacenar los valores) y en las consultas (*joins* por punteros).
- Los autores reconocen que la frontera se difumina: los SGBD orientados a objetos empezaban a guardar objetos en memoria con punteros directos (*swizzling*). Prevén que en el futuro todo buen SGBD reconocerá que hay datos residentes en memoria y los gestionará de forma distinta.

2.3 ¿Qué había que mantener de los sistemas convencionales para evitar pérdidas ante fallos?

El **registro de transacciones (log)** y una **copia de respaldo en disco** de la base de datos, actualizada mediante **checkpoints**. Para que una transacción sea duradera, su registro de log debe llegar a almacenamiento estable antes de confirmarse. El artículo propone aligerar ese coste con memoria estable para la cola del log, *pre-commit* y *group commit*. Tras un fallo, la base se reconstruye cargando la copia de respaldo (completa o por segmentos bajo demanda) y aplicando el log. Como la memoria es más frágil, los respaldos deben hacerse con más frecuencia que en una DRDB, lo que convierte el rendimiento del mecanismo de copia en un aspecto central.

2.4 ¿Alguna de las bases de datos comentadas ha sobrevivido?

De los siete sistemas revisados (OBE, MM-DBMS, IMS/VS Fast Path, MARS, HALO, TPK y System M), **sólo IMS Fast Path sigue en uso**. IMS es el gestor jerárquico de IBM para mainframe. Sus bases de datos *Data Entry Databases* (DEDB) y *Main Storage Databases* (MSDB), heredadas de Fast Path, siguen existiendo en las versiones actuales de IMS y lo usan grandes bancos y aseguradoras. El resto eran **prototipos de investigación** (IBM, Universidad de Wisconsin, Southern Methodist University, Princeton) que no llegaron a producto, aunque sus ideas sí sobrevivieron: los T-trees, la recuperación por segmentos, los *checkpoints* difusos, el *group commit* y la ejecución serial de transacciones de TPK. Esta última reaparece en sistemas modernos como VoltDB, y también en Redis, que es monohilo.

Conclusiones propias. El artículo anticipa con acierto lo que ha pasado después: la memoria se ha abaratado en órdenes de magnitud y hoy SAP HANA, Oracle TimesTen/In-Memory, SQL Server In-Memory OLTP, Redis o el motor inMemory de MongoDB son productos comerciales. Los problemas que plantea siguen siendo los mismos: la volatilidad se resuelve con logs, réplicas y *snapshots*, y el tamaño del conjunto caliente sigue condicionando la arquitectura. Lo que en 1992 era una idea de futuro limitada por el hardware es hoy un componente estándar de las arquitecturas de datos.

E3. Caso de uso real: Axis Bank con Redis

Aspecto	Detalle
Empresa	Axis Bank (Bombay, India): tercer banco privado del país por activos, con más de 10 millones de usuarios diarios en su banca móvil.
Problema	La app móvil no reflejaba en tiempo real los cambios hechos en sucursal (productos contratados, personas autorizadas…), porque leía de bases de datos tradicionales sin sincronización en tiempo real. Eso generaba quejas de los clientes.
Plataforma elegida	**Redis Enterprise** como capa de datos operativa en memoria, con **Redis Data Integration (RDI)** para ingerir los cambios, **Redis Query Engine** para consultas concurrentes, *pipelining* y alta disponibilidad con réplicas primaria/secundaria gestionadas con Pacemaker.
Arquitectura	Las tablas del *core* bancario se replican con Oracle GoldenGate a una base local. RDI captura los cambios, filtra los registros irrelevantes y los transforma en *hashes* de Redis. La app consulta en paralelo 9 tablas en Redis en lugar de hacerlo contra el *core*.
Objetivos	Mostrar la información de la cuenta actualizada y disponible 24/7, sin intervención manual, y reducir los tiempos de respuesta.
Mejoras	Transacciones un **76 % más rápidas**: la espera del usuario baja de **170 ms a 40 ms** (4,25 veces más rápido que consultar el *core*). Ahorro de unos **82.000 USD** al almacenar sólo los registros necesarios, y mayor estabilidad para más de 10 millones de usuarios diarios.
Objetivos futuros	El caso publicado no concreta una hoja de ruta. Presenta la solución como base para seguir ofreciendo información actualizada 24/7 al crecer el número de usuarios (de 6 a más de 10 millones diarios) sin intervención manual.

Valoración. Es un uso típico de una base de datos en memoria como capa de servicio rápida delante de un sistema de registro lento: no sustituye al *core*, sino que lo descarga de lecturas intensivas y sirve datos de baja latencia. Encaja con lo visto en la asignatura. La persistencia real sigue en el sistema origen, la sincronización se hace por captura de cambios (CDC) y la alta disponibilidad compensa la volatilidad de la memoria. Fuente: Redis, caso de cliente Axis Bank (https://redis.io/customers/axis-bank/).

E4. Activación del motor IN-MEMORY de MongoDB

El motor *inMemory* sólo está disponible en **MongoDB Enterprise**; no existe en la Community. Se ha usado MongoDB Enterprise Server 8.3.11, la misma instalación de la actividad de bases de datos documentales. No hace falta instalar nada adicional: basta con indicar a `mongod` el motor de almacenamiento al arrancar. Hay dos formas:

- **Por línea de comandos:** `mongod --storageEngine inMemory --inMemorySizeGB 1 --dbpath <ruta>`. `--inMemorySizeGB` fija la memoria máxima para datos e

índices; por defecto es el 50 % de la RAM menos 1 GB. El `dbpath` sigue siendo obligatorio para ficheros de diagnóstico, pero los datos no se escriben en él.

- **Por fichero de configuración** (`mongod.conf`): `storage.engine: inMemory` y `storage.inMemory.engineConfig.inMemorySizeGB`. Después: `mongod --config mongod.conf`.
- En Windows es igual con `mongod.exe`. Si MongoDB está instalado como servicio, hay que editar el `mongod.cfg` del servicio y reiniciarlo. Un `dbpath` no puede contener datos de otro motor, así que conviene usar una carpeta vacía.

Se levanta una segunda instancia en el puerto 27018 con el motor en memoria y se comprueba con `db.serverStatus().storageEngine` que el motor es `inMemory` y `persistent: false`. `getCmdLineOpts` muestra la configuración efectiva:

```
aramber@localhost:~$ mongod --storageEngine inMemory --inMemorySizeGB 1 --dbpath /data/mongo-mem --port 27018 --bind_ip 127.0.0.1 --logpa
th /data/mongo-mem/mongod.log --fork 2>&1 | grep -v Shuffling
about to fork child process, waiting until server is ready for connections.
forked process: 18435
child process started successfully, parent exiting
aramber@localhost:~$ mongosh --quiet --port 27018 --eval 'db.serverStatus().storageEngine'
{
  name: 'inMemory',
  supportsCommittedReads: true,
  oldestRequiredTimestampForCrashRecovery: Timestamp({ t: 0, i: 0 }),
  dropPendingIdents: Long('0'),
  dropSpillTableRetries: Long('0'),
  supportsSnapshotReadConcern: true,
  readOnly: false,
  persistent: false,
  backupCursorOpen: false
}
aramber@localhost:~$ mongosh --quiet --port 27018 --eval 'db.adminCommand({getCmdLineOpts:1}).parsed.storage'
{
  dbPath: '/data/mongo-mem',
  engine: 'inMemory',
  inMemory: { engineConfig: { inMemorySizeGB: 1 } }
}
aramber@localhost:~$ mongosh --quiet --port 27018 --eval 'db.getSiblingDB("mbda").towns.insertOne({name:"Sevilla", population: 684025});
db.getSiblingDB("mbda").towns.find()'
[
  {
    _id: ObjectId('6aae5faa4abaa19d7b8313dc'),
    name: 'Sevilla',
    population: 684025
  }
]
```

Figura 1. Arranque de mongod con el motor inMemory y comprobaciones.

Para demostrar que no hay persistencia, se inserta un documento, se detiene la instancia y se vuelve a arrancar, esta vez mediante fichero de configuración. La colección está vacía (`countDocuments()` = 0): todo lo que había en memoria se ha perdido. También se ve que esta instancia rechaza las escrituras con `j:true` porque no tiene *journal* (evidencia en la actividad In-Memory II).

```
aramber@localhost:~$ cat mongod-inmemory.conf
# /etc/mongod-inmemory.conf  — ejemplo de configuración equivalente
storage:
  dbPath: /data/mongo-mem
  engine: inMemory
  inMemory:
    engineConfig:
      inMemorySizeGB: 1
net:
  port: 27018
  bindIp: 127.0.0.1
systemLog:
  destination: file
  path: /data/mongo-mem/mongod.log
processManagement:
  fork: true
aramber@localhost:~$ mongod --config mongod-inmemory.conf 2>&1 | grep -v Shuffling | tail -1
child process started successfully, parent exiting
aramber@localhost:~$ mongosh --quiet --port 27018 --eval 'db.getSiblingDB("mbda").towns.countDocuments()'
0
```

Figura 2. Fichero de configuración equivalente, rearranque y comprobación de que los datos no persisten.

E5. Instalación de REDIS

La guía oficial (https://redis.io/download) ofrece tres vías en Linux: paquetes del sistema (`apt`/`snap`), el repositorio APT oficial `packages.redis.io` o compilar desde el código fuente con `make`. Se ha elegido **el paquete APT de Ubuntu 24.04** (`redis-server` 7.0.15) por sencillez y porque deja Redis integrado con el sistema. La contrapartida es que no es la última versión: el repositorio oficial ofrece 7.4/8.x. En Windows no hay versión nativa soportada y habría que usar WSL2 o Docker.

```
aramber@localhost:~$ apt-get update -qq 2>/dev/null; apt-get install --reinstall -y redis-server redis-tools 2>/dev/null | grep -E 'Get:|
Unpacking|Setting up'
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 redis-tools amd64 5:7.0.15-1ubuntu0.24.04.4 [1166
kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 redis-server amd64 5:7.0.15-1ubuntu0.24.04.4 [51.7
 kB]
Unpacking redis-tools (5:7.0.15-1ubuntu0.24.04.4) over (5:7.0.15-1ubuntu0.24.04.4) ...
Unpacking redis-server (5:7.0.15-1ubuntu0.24.04.4) over (5:7.0.15-1ubuntu0.24.04.4) ...
Setting up redis-tools (5:7.0.15-1ubuntu0.24.04.4) ...
Setting up redis-server (5:7.0.15-1ubuntu0.24.04.4) ...
aramber@localhost:~$ redis-server --version
Redis server v=7.0.15 sha=00000000:0 malloc=jemalloc-5.3.0 bits=64 build=e53ff17674aa6190
aramber@localhost:~$ redis-server --daemonize yes --port 6379 && sleep 1 && redis-cli ping
PONG
aramber@localhost:~$ redis-cli INFO server | sed -n '2,3p;7p;19,20p'
redis_version:7.0.15
redis_git_sha1:00000000
os:Linux 6.18.44-fc-v37 x86_64
uptime_in_days:0
hz:10
```

Figura 3. Instalación con `apt-get`, versión instalada, arranque como demonio y prueba de conexión (`PING` → `PONG`).

Decisiones y dificultades. (1) En el contenedor de pruebas no hay `systemd`, así que en lugar de `systemctl start redis-server` se lanza el servidor como demonio con `redis-server --daemonize yes`. (2) Por defecto Redis escucha sólo en `127.0.0.1`, lo adecuado en un entorno de pruebas. Para exponerlo habría que cambiar `bind` y activar `requirepass`. (3) Aunque es una base de datos en memoria, Redis **persiste por defecto** con *snapshots* RDB según la política `save 3600 1 300 100 60 10000` (figura siguiente). Se puede activar AOF (`appendonly yes`) para mayor durabilidad o desactivar toda persistencia (`save ""`) si se usa sólo como caché. Se ha probado el funcionamiento con operaciones básicas de clave-valor, contadores atómicos, *hashes* y claves con caducidad (TTL):

```
aramber@localhost:~$ redis-cli SET asignatura 'GVDM'
OK
aramber@localhost:~$ redis-cli GET asignatura
GVDM
aramber@localhost:~$ redis-cli INCR visitas; redis-cli INCR visitas
1
2
aramber@localhost:~$ redis-cli HSET alumno:1 nombre 'Alumno MBDA' curso 2025-26
2
aramber@localhost:~$ redis-cli HGETALL alumno:1
nombre
Alumno MBDA
curso
2025-26
aramber@localhost:~$ redis-cli SET sesion:abc 'token' EX 60 && redis-cli TTL sesion:abc
OK
60
aramber@localhost:~$ redis-cli INFO memory | grep -E 'used_memory_human|maxmemory_human|mem_allocator'
used_memory_human:1.06M
maxmemory_human:0B
mem_allocator:jemalloc-5.3.0
aramber@localhost:~$ redis-cli CONFIG GET save
save
3600 1 300 100 60 10000
```

Figura 4. Prueba de Redis: SET/GET, INCR, HSET/HGETALL, expiración con TTL, uso de memoria y política de persistencia.

Referencias

[1] K. Aschenbrenner, «Why I don’t (yet) recommend In-Memory OLTP to my customers», SQLpassion, 11/03/2015. https://www.sqlpassion.at/archive/2015/03/11/dont-yet-recommend-memory-oltp-customers/

[2] K. Aschenbrenner, «Why I now (eventually) recommend In-Memory OLTP», SQLpassion, 16/11/2015.

[3] H. García-Molina y K. Salem, «Main Memory Database Systems: An Overview», IEEE TKDE, 4(6), 509-516, 1992.

[4] Redis, «Axis Bank» (caso de cliente). https://redis.io/customers/axis-bank/

[5] MongoDB Manual – In-Memory Storage Engine. https://www.mongodb.com/docs/manual/core/inmemory/

[6] Redis – Install Redis on Linux. https://redis.io/docs/latest/operate/oss_and_stack/install/